Bézier Curve Series

Overview and Reader's Guide

Julia + GLMakie + Symbolics.jl

`Scripts 01-35`

E.R. Nurwijayadi

2026-03-15

Directed and curated by E.R. Nurwijayadi. Code and explanations AI-generated.

> *Bézier curves are everywhere: the rounded corners of every app button, the letterforms of every font on your screen, the smooth swoop of a car body, the easing on every animation. You interact with them dozens of times a day without noticing.*
> *After working through this series, you will notice them every time — and know exactly how they work.*
> *Why did the control point feel left out? Because the curve never passes through it. (That is the only joke. Everything that follows is serious.)*

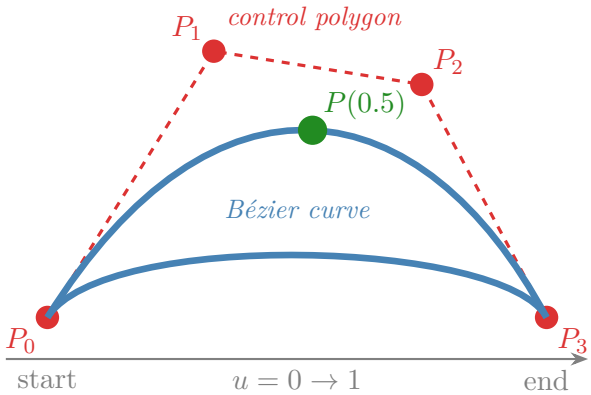
Contents

1. What Is a Bézier Curve?

1.1. The geometric idea

A Bézier curve is a smooth parametric curve defined by a small set of **control points**. The curve passes through the first and last control points (the endpoints) but is only *attracted* by the intermediate ones — it bends toward them without touching them.

The curve is traced by a parameter u that sweeps from 0 to 1. At $u = 0$ you are at the first control point. At $u = 1$ you are at the last. At every u in between you are somewhere on the smooth curve connecting them.



The **control polygon** (dashed red) is the skeleton. The **Bézier curve** (solid blue) is the smooth result. The green dot shows the curve point at $u = 0.5$.

1.2. The convex hull property

Every point on a Bézier curve is a weighted average of the control points, where all weights are non-negative and sum to 1. This means the curve is always contained inside the convex hull of its control points — the smallest convex shape that encloses all of them.

This property has a practical consequence: if you move a control point, only the part of the curve near that point changes. The rest is unaffected.

1.3. Degree and number of control points

Name	Degree	Control points	Curve shape
Linear	1	2	Straight line
Quadratic	2	3	Single arc
Cubic	3	4	S-curve possible
Quartic	4	5	More complex shapes
Degree n	n	$n + 1$	General

This series works with quadratic (scripts 11–12) and quartic (scripts 21–22) as concrete cases, then generalises to any degree (scripts 31–35).

2. The Two Evaluation Methods

Given a set of control points and a parameter u, how do you find the point on the curve? The series explores two methods that always give the same answer but work in completely different ways.

2.1. Method 1: Bernstein polynomials

The Bernstein method computes a weighted average of all control points simultaneously:

$$P(u) = \sum_{i=0}^{n} B(n, i, u) \cdot P_i \qquad B(n, i, u) = \binom{n}{i} u^i (1-u)^{n-i}$$

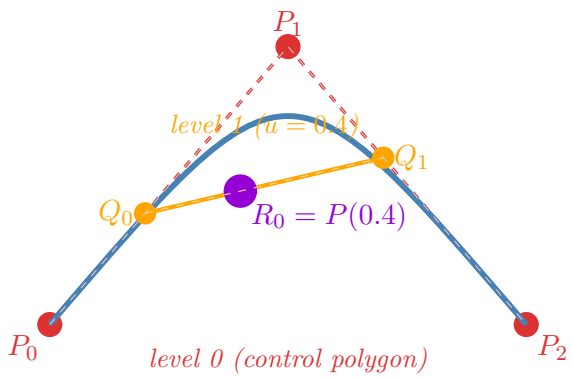
Each $B(n, i, u)$ is a weight that tells you how much influence control point P_i has at parameter u. All weights are non-negative and sum to 1. At $u = 0$, only P_0 has weight 1; all others are 0. At $u = 1$, only P_n has weight 1. In between, the weights blend smoothly.

> **Quadratic ($n = 2$) at $u = 0.5$:**
>
> $B(2, 0, 0.5) = 0.25, \quad B(2, 1, 0.5) = 0.50, \quad B(2, 2, 0.5) = 0.25$
>
> $P(0.5) = 0.25P_0 + 0.50P_1 + 0.25P_2$
>
> The midpoint control point P_1 contributes twice as much as the endpoints — the curve is pulled toward it, but only reaches halfway.

2.2. Method 2: De Casteljau algorithm

The De Casteljau method is purely geometric. Instead of computing weights, it repeatedly cuts line segments.

At each step, slide fraction u along every edge of the current polygon to get a new, shorter polygon. After n steps for a degree-n curve, you are left with a single point — the curve point at parameter u.



The orange segment Q_0Q_1 is level 1: each Q point is 40% of the way along its parent edge. The purple point R_0 is 40% of the way along Q_0Q_1 — it sits exactly on the curve.

2.3. Which method does the series use where?

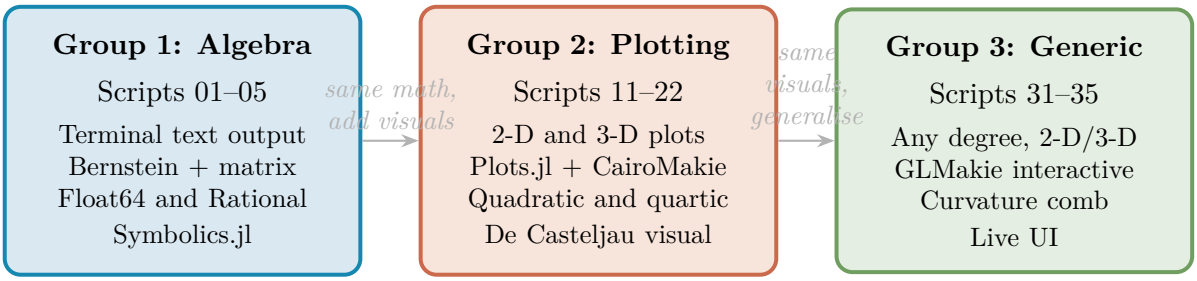
Method	Scripts	Why
Bernstein (algebra)	01–05	Exact polynomial form
Bernstein (plotting)	11, 21, 31, 32	Efficient for many points
De Casteljau	12, 22, 31–35	Geometric insight, visualisation
Both together	31–35	Comparison and curvature

Both methods are implemented in parallel throughout the series. They always produce the same curve — the series uses both precisely to demonstrate this equivalence.

3. The Series at a Glance

3.1. Three groups, one progression

The 15 scripts fall naturally into three groups:



3.2. All 15 scripts

Script	Name	Output	Key new concept
examplebg 01	bezier-algebra	Text	Hand-rolled `Poly`, multiple dispatch
02	bezier-sympy	Text	`Rational{Int}`, Symbolics.jl
examplebg 03	bezier-matrix-pedagogical	Text	$U \cdot M \cdot G$, step-by-step
04	bezier-matrix-straight	Text	Section-based output
examplebg 05	bezier-matrix-sympy-text	Text	Symbolic + rational combined
11p	bezier-quadratic-plots	2-D plot	Plots.jl, `hcat+splat+transpose`
examplebg 11m	bezier-quadratic-makie	2-D plot	CairoMakie, Figure/Axis split
12p	de-casteljau-plots	2-D plot	`lerp`, named tuple, snapshots
examplebg 12m	de-casteljau-makie	2-D plot	`titlegap`, legend control
21	bezier-quartic-makie	3-D static	`Axis3`, elevation/azimuth
examplebg 22	de-casteljau-makie (3D)	3-D static	`getfield`, NaN dummy lines
31	bezier-generic	2-D/3-D	GLMakie, `is_3d` branch, generic DC
examplebg 32	bezier-symbolic	2-D/3-D	`build_symbolic_poly`, substitution
34	bezier-curvature	2-D/3-D	Derivative control points, comb
examplebg 35	bezier-interactive	UI	`Observable`, sliders, textboxes

Why is there no script 33? The numbering follows the Python source series from which the Julia scripts were ported. Script 33 exists in the original series but was not included here.

4. How to Read These Documents

4.1. Document naming

Each script has one or more explanation documents. The naming convention is:

Filename pattern	Contents
`NN-name-code-explained.tex`	Code explanation only (scripts 01–05)
`NN-name-output-explained.tex`	Output explanation only (scripts 01–05)
`NN-name-explained.tex`	Merged: plot first, then code (scripts 11–35)
`40-bezier-code-analysis.tex`	Series-wide architecture and pattern analysis
`00-bezier-overview.tex`	This document

4.2. Box colour meanings

Four coloured boxes appear throughout the documents. Each has a consistent meaning:

Colour	Name	Used for
examplebg Blue	Example box	Concrete worked examples with actual numbers. Traces through what the code computes.
notebg Yellow	Note box	Warnings, caveats, and "why" explanations. Things that would bite a beginner.
diffbg Green	Insight box	Design insights, patterns, and broader context. The bigger picture.
samebg Grey	Reference box	"This was explained earlier." Points to the prior document rather than repeating.

4.3. Typographic conventions

Convention	Meaning
`monospace`	Julia code, function names, variable names, type names, filenames
$P(u)$, κ, u	Mathematical quantities
bold	First use of an important term
italic	Emphasis or a note to the reader

4.4. Each document is self-contained

Every explanation document can be read independently. Where a concept was introduced in a prior script, a grey reference box points back to it rather than repeating the full explanation. This means you can start at any script without having read all the previous ones — though of course reading in order gives the clearest picture.

4.5. Data tables

The plotting documents (scripts 11–35) include concrete data tables showing the actual values of key variables such as the curve matrix `cpts`, the control polygon matrix `cp`, the highlighted points `hpts`, and the individual point vector `pt`. These ground the abstract code in real numbers.

5. Setting Up

5.1. Installing Julia packages

All required packages must be installed once before running any script. Open a Julia REPL and run:

<MINTED>

Package	Used in	Purpose
`Symbolics`	02, 05, 32	Symbolic mathematics
`Plots`	11p, 12p	Static plots (GR backend)
`CairoMakie`	11m, 12m, 21, 22	Static plots (Cairo backend)
`GLMakie`	31, 32, 34, 35	Interactive GPU-accelerated plots

`Printf`, `LinearAlgebra`, and `Statistics` are part of Julia's standard library and require no installation.

5.2. Running a script

From the terminal, in the directory containing the scripts:

<MINTED>

Scripts that open a plot window (all GLMakie scripts and Plots.jl scripts with `display`) will wait for you to press Enter before closing. Scripts 01–05 print to the terminal only — no window.

5.3. The `CASE` switch

Scripts 01–05, 31, 32, and 34 contain a single line near the top:

<MINTED>

This is the **only line you need to change** to switch between the two pre-configured cases. Everything else — control points, output filename, u highlight values — is set automatically based on the case number.

GLMakie requires a display. Scripts 31, 32, 34, and 35 use GLMakie for interactive or rotatable 3-D plots. They require a desktop environment with a GPU-capable display (X11 or Wayland on Linux, native on macOS/Windows). On a headless server (no screen), use the CairoMakie versions (scripts 11m, 12m, 21, 22) instead.

5.4. What each script produces

Script(s)	Output	Notes
01–05	Terminal text	No files saved
11p, 12p	`*.png` + window	Saved by `savefig`
11m, 12m, 21, 22	`*.png` + window	Saved by `save`
31, 32, 34	`*.png` + window	One PNG per case
35	Window only	Interactive; no automatic save

6. Reading Guide by Goal

Not everyone needs to read the series front to back. Here is where to start depending on what you want to learn.

Goal	Start here	Then continue
Understand the Bézier polynomial	01-bezier-algebra	02, 03, 04
See exact rational arithmetic	02-bezier-sympy	05
Understand the matrix method	03-bezier-matrix-pedagogical	04, 05
First 2-D plot in Plots.jl	11-bezier-quadratic-plots	12-plots
First 2-D plot in Makie	11-bezier-quadratic-makie	12-makie
See De Casteljau visualised	12-de-casteljau-plots	12-makie, 22
Work with 3-D curves	21-bezier-quartic-makie	22, 31
Write generic code for any degree	31-bezier-generic	32
Combine Symbolics with plotting	32-bezier-symbolic	(end of series)
Understand curvature	34-bezier-curvature	35
Build an interactive UI in Julia	35-bezier-interactive	40 (analysis)
Understand the full architecture	40-bezier-code-analysis	anywhere

The fastest path to a working 3-D interactive plot:

1. Read `21-bezier-quartic-makie-explained` to understand `Axis3`, elevation, azimuth, and 3-D drawing.

2. Read `31-bezier-generic-explained` to see how one script handles both 2-D and 3-D.

3. Run `31-bezier-generic.jl` with `CASE = 2`. Rotate the 3-D window.

Those three documents plus one run will give you a solid foundation for the interactive script (35) and the curvature extension (34).

7. Julia Concepts You Will Encounter

If you are new to Julia, this is a preview of the language features the series introduces, in roughly the order they first appear.

Concept	What it means	First seen
`struct` + constructor	Custom types with computed fields	Script 01
Multiple dispatch	Same function name, different behaviour by type	Script 01
`const` + `Dict`	Immutable module-level data	Scripts 01, 11
Broadcast `.* .+`	Element-wise operation on arrays	Script 11
`hcat` + `splat` + `transpose`	Build a matrix from a list of vectors	Script 11
Named tuple return	`return (field=val, ...)`	Script 12
`LinRange`	Lazy evenly-spaced range	Script 11
`Axis3`, `elevation`, `azimuth`	3-D camera control in Makie	Script 21
`getfield(obj, :sym)`	Dynamic field access by symbol	Script 22
`while` loop with `push!`	Accumulate results into a vector	Script 31
`Union{T, Nothing}` + `isnothing`	Optional typed fields	Script 34
`diff(A, dims=1)`	Row-difference shortcut	Script 34
`Observable` vs `Ref`	Notifying vs non-notifying mutable cell	Script 35
`on(obs) do ... end`	Reactive callback	Script 35
`let` closure capture	Freeze loop variable for callback	Script 35
`tryparse(T, s)`	Safe string conversion	Script 35

None of these requires prior Julia experience to understand. Each is explained from scratch in the document where it first appears.